

Contents

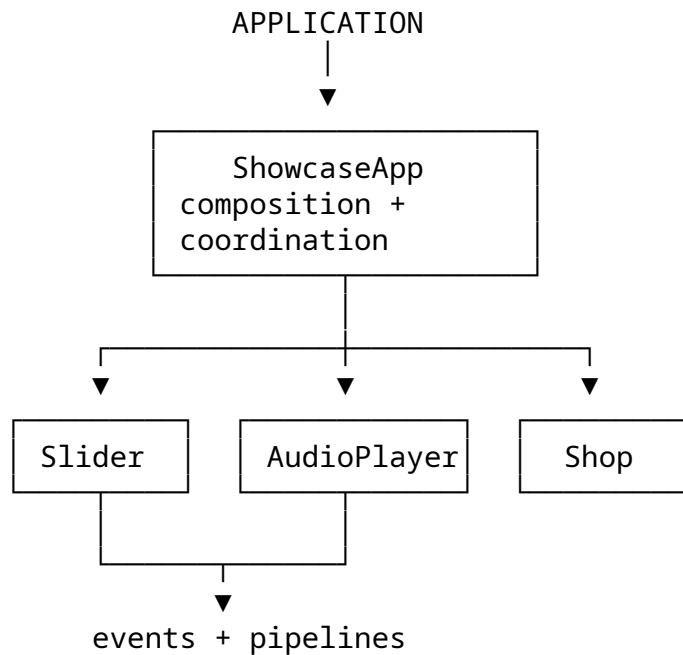
Интерактивная музыкальная витрина - архитектура с первого взгляда . . .	2
1. Центральная идея	2
1.1 Slider растёт по возможностям	2
1.2 Три локальных FSM	3
1.3 Менеджеры образуют переиспользуемый инфраструктурный слой	3
1.4 Ввод проходит через pipeline	3
1.5 Бесконечный цикл и drag	4
1.6 Архитектурный словарь	4
1.7 Почему архитектура намеренно достаточно масштабная	4

Интерактивная музыкальная витрина - архитектура с первого взгляда

Нативный JavaScript-проект с архитектурой в центре внимания

Это проект, в котором архитектура важнее самой визуальной задачи. На поверхности это витрина альбомов, слайдер и аудиоплеер, но основной предмет работы находится глубже: изоляция компонентов, последовательное расширение через наследование, локальные конечные автоматы, переиспользуемые менеджеры, событийная координация и легковесный pipeline ввода.

Одна и та же архитектура реализована дважды - с prototype-based OOP и с ES6 classes. Поэтому документация описывает общую систему, а не привязывает её к одному синтаксису.



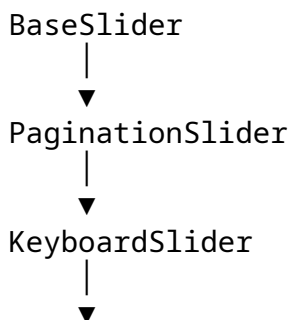
Центральная идея

Проект строится вокруг одного правила владения ответственностью:

Компоненты владеют доменным поведением и локальным состоянием.
Менеджеры владеют переиспользуемыми механизмами взаимодействия.
ShowcaseApp владеет композицией и политикой межкомпонентной координации.
События соединяют компоненты, не раскрывая их внутреннее устройство.

Это важнее любого отдельного паттерна.

Slider растёт по возможностям



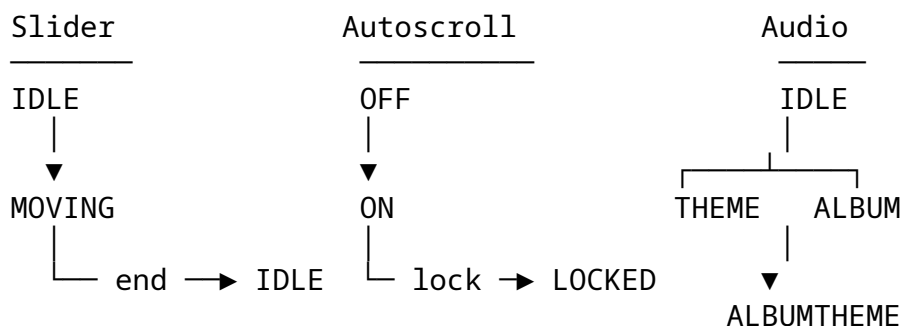
```

InfiniteSlider
  |
  ▼
DraggableSlider
  |
  ▼
AutoscrollSlider

```

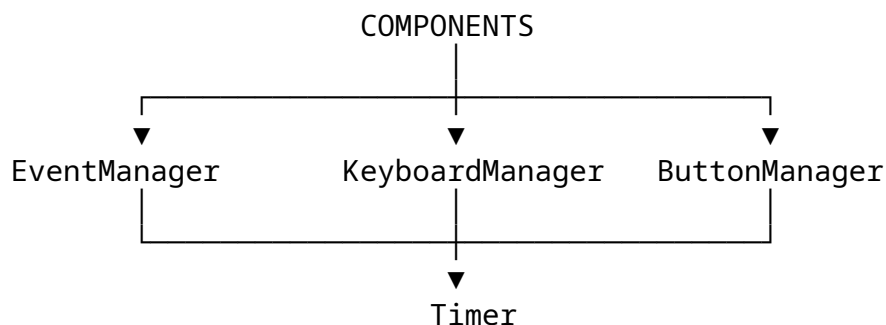
Финальный Slider не является набором несвязанных функций. Каждый уровень добавляет одну осмысленную возможность и использует контракт предыдущего уровня.

Три локальных FSM



Состояния намеренно локальны. Здесь нет искусственного глобального объекта State, потому что каждое состояние принадлежит компоненту, который его понимает.

Менеджеры образуют переиспользуемый инфраструктурный слой



DOMValidator = DOM contract
 Button = command-like wrapper

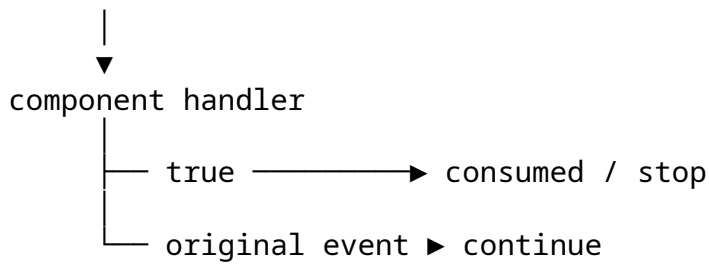
Менеджеры нужны для того, чтобы вынести универсальные механизмы из доменных компонентов. KeyboardManager не знает, что такое альбом; EventManager не знает, что такое drag.

Ввод проходит через pipeline

```

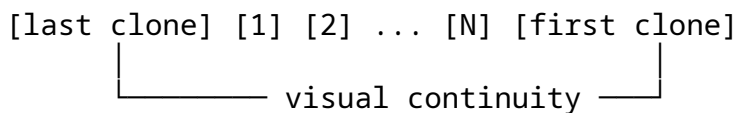
Browser event
  |
  ▼
ShowcaseApp
  |
  ▼
Keyboard/Button manager

```



Это легковесный протокол в духе Chain of Responsibility, реализованный через возвращаемые значения, а не через большой фреймворк абстракций.

Бесконечный цикл и drag



Drag lifecycle:

pointer down -> dynamic subscriptions -> drag -> threshold -> next/prev/no-op ->

Остальная система не должна знать, что существуют clone-слайды. Аналогично, drag-слушатели живут только во время самого drag.

Архитектурный словарь

Mediator-like	-> ShowcaseApp
Observer/Pub-Sub-like	-> custom events + EventManager
Chain of Responsibility	-> _pipe / handler propagation
Strategy-like	-> configurable managers
Template Method-like	-> slider inheritance hooks
Command-like	-> Button
FSM	-> local state models
DI / IoC	-> composition and initialization
Polymorphism	-> slider hierarchy + selected static contracts
Fail-fast validation	-> DOMValidator / config checks

Эти названия описывают реальные структурные идеи. Проект намеренно не пытается подогнать каждый механизм под textbook GoF-паттерн.

Почему архитектура намеренно достаточно масштабная

Целью не было написать минимально возможный Slider. Цель состояла в том, чтобы сделать архитектуру достаточно понятной, контролируемой и переиспользуемой для изучения:

границы компонентов
 +
 наследование / полиморфизм
 +
 FSM
 +
 менеджеры
 +

событийные pipeline
+
конфигурация
+
явные контракты